

TRABALHO 1 - DIAGNÓSTICO DE PROCESSOS EM LINUX

Análise do processo Firefox

Alunos	Brenno Patrcik Galvão Pereira/ Dmylle Charis de Andrade Santos
Disciplina	Sistemas Operacionais - 2 ADS A 2026/2
Ambiente	Ubuntu Linux - versão: 26.04.1
Tipo de ambiente	Máquina física
Link do Git	https://github.com/Amyi1/DIAGNOSTICO-DE-PROCESSOS-EM-LINUX

Objetivo:

O objetivo deste trabalho foi acompanhar um processo real no Linux e observar como o sistema operacional identifica, organiza e controla esse processo. O Firefox foi escolhido por ser uma aplicação de usuário que pode ser monitorada, pausada, retomada e encerrada com segurança.

1. Identificação do processo - PID e PPID

Na coleta principal, o Firefox foi localizado com `pgrep firefox` e recebeu o PID 8976.

```
$ pgrep firefox
8976

$ ps -o pid,ppid,stat,pri,ni,%cpu,%mem,cmd -p 8976
PID   PPID  STAT PRI NI   %CPU %MEM  CMD
8976  2174  Sl   19  0    2.4  2.5   /snap/firefox/8763/usr/lib/firefox/firefox

$ ps -p 2174 -o pid,ppid,stat,cmd
PID   PPID  STAT  CMD
2174  1706  Rsl    /usr/bin/gnome-shell --mode=ubuntu
```

O PID é o número que identifica o processo. Já o PPID indica qual era o processo pai naquele momento. Na coleta, o PPID 2174 correspondia ao GNOME Shell, formando a relação GNOME Shell (2174) -> Firefox (8976).

2. Árvore de processos

A árvore foi consultada com `pstree -p`. O Firefox apareceu abaixo do GNOME Shell e apresentou vários processos/componentes associados.

```
systemd(1) `--
systemd(1706)
  `-- gnome-shell(2174)
    `-- firefox(8976)
        |-- forkserver
        |-- RDD Process
        |-- Socket Process
        |-- Utility Process
        |-- Web Content
        `-- WebExtensions
```

Com essa saída, conseguimos ver a hierarquia dos processos e que o Firefox possui vários componentes para realizar suas tarefas.

3. Estado do processo

Na coleta inicial, o campo STAT apresentou Sl. A letra S indica que o processo estava em espera (sleeping), aguardando algum evento. A letra l indica que o processo possui múltiplas threads. Esse estado é esperado enquanto o Firefox aguarda algum evento.

4. Uso de CPU e memória

Coleta	CPU	Memória
1º momento	2,4%	2,5%

2º momento	2,0%	3,6%
Outra coleta do log	3,9%	6,7%

Os valores mudaram entre as coletas porque o consumo do Firefox depende do que estava acontecendo no navegador naquele momento.

5. Prioridade e nice

Inicialmente, o processo apresentou PRI 19 e NI 0. Depois foi realizada uma alteração controlada com renice.

```
Antes:
PID    PRI NI
8976   19  0

$ renice 5 -p 8976
8976 (process ID) old priority 0, new priority 5

Depois:
PID    PRI NI
8976   14  5
```

O nice passou de 0 para 5. Um valor de nice maior representa menor prioridade relativa na disputa pelo uso da CPU. A consulta feita depois confirmou a alteração.

6. Threads

O comando `ps -L -p 8976` mostrou várias LWPs associadas ao Firefox. Entre os nomes observados estavam Socket Thread, HTML5 Parser, JS Watchdog, Renderer, Compositor, ImageIO e outras.

Além disso, `/proc/8976/status` registrou Threads: 83. O processo representa a aplicação em execução, enquanto as threads são unidades de execução dentro do processo e compartilham seus recursos.

7. Análise do /proc/8976

Fonte	Informação observada
<code>/proc/8976/status</code>	Estado S, PPID 2174, VmRSS 151416 kB e 83 threads.
<code>/proc/8976/limits</code>	Limites de recursos, como pilha, processos e arquivos abertos.
<code>/proc/8976/cmdline</code>	<code>/snap/firefox/8763/usr/lib/firefox/firefox</code>
<code>/proc/8976/io</code>	Contadores de leitura e escrita do processo.

Esses arquivos foram úteis porque permitiram consultar informações do processo diretamente pelo sistema Linux. No arquivo de I/O, por exemplo, foram registrados 527943090 em `rchar`, 293008983 em `wchar`, 401465344 em `read_bytes` e 291762176 em `write_bytes`.

8. Controle por sinais

Na primeira execução, os sinais foram testados no PID 8976. Como a consulta de estado não tinha sido registrada entre o SIGSTOP e o SIGCONT, foi feita uma coleta complementar. O Firefox foi aberto novamente e recebeu o PID 14693. Essa segunda coleta foi usada somente para completar a evidência antes/depois dos sinais.

Antes do SIGSTOP:

```
$ ps -o pid,ppid,stat,pri,ni,%cpu,%mem,cmd -p 14693
PID    PPID STAT PRI NI %CPU %MEM CMD
14693   2174 S1    19  0  27.8  5.1 /snap/firefox/8763/usr/lib/firefox/firefox
```

Após o SIGSTOP:

```
$ kill -STOP 14693
$ ps -o pid,ppid,stat,pri,ni,%cpu,%mem,cmd -p 14693
PID    PPID STAT PRI NI %CPU %MEM CMD
14693   2174 T1    19  0  13.7  3.7 /snap/firefox/8763/usr/lib/firefox/firefox
```

O estado mudou de S1 para T1. A letra T comprova que o processo estava suspenso após o SIGSTOP.

Após o SIGCONT:

```
$ kill -CONT 14693
$ ps -o pid,ppid,stat,pri,ni,%cpu,%mem,cmd -p 14693
PID    PPID STAT PRI NI %CPU %MEM CMD
14693  2174 Rl   19  0  11.5  3.7 /snap/firefox/8763/usr/lib/firefox/firefox
```

Depois do SIGCONT, o estado passou para Rl. A letra R indica que o processo estava executando ou pronto para execução, confirmando a retomada.

Encerramento com SIGTERM:

```
$ kill -TERM 14693
$ sleep 2
$ ps -p 14693
PID TTY          TIME CMD
```

Como o comando ps mostrou somente o cabeçalho e nenhuma linha para o PID 14693, o processo já havia sido encerrado. Diferente do SIGSTOP, que apenas suspende, o SIGTERM solicita o término do processo.

9. Conclusão técnica

Com essa atividade foi possível entender melhor como funcionam os processos no Ubuntu usando o Firefox como exemplo. Foi possível ver o PID e o PPID, o uso de CPU e memória, a prioridade do processo, as threads e também algumas informações pelo /proc. Também testamos alguns sinais no processo do Firefox. No começo ele estava no estado Sl, depois do SIGSTOP mudou para Tl e com o SIGCONT voltou para Rl. No final foi usado o SIGTERM para encerrar o processo e depois vimos que o PID não aparecia mais. No geral, a atividade ajudou a entender na prática alguns assuntos que vimos sobre processos em Sistemas Operacionais e como podemos acompanhar e controlar eles pelo terminal.